```
root@do-manager-1:~# docker swarm init --advertise-addr 46.101.90.10
Swarm initialized: current node (e2gaylk0nnr1geabo40cn3uf9) is now a manager

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0axry1rp0wxy6u48t4epiml4mubf9qy2y2o2f
dmq1u7n2vnj08-aoqoyn55bypasn82emb3sae27 46.101.90.10:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and
follow the instructions.
```

Now that we've set up our swarm manager instance, we can exit from the box:

```
root@do-manager-1:~# exit
logout
```

For redundancy and resiliency in production environments, you'd typically run your containers on a cluster with multiple instances. This way, if a single node went down, the containers lost can be started up again on the remaining instances. We're going to have a three-node cluster. To achieve this, let's create two *worker* instances as follows:

```
$ docker-machine create \
  --driver digitalocean \
  --digitalocean-access-token $DIGITAL_OCEAN_TOKEN \
  --digitalocean-region lon1 \
  do-worker-1
Running pre-create checks...
Creating machine...
«...»
Docker is up and running!
To see how to connect your Docker Client to the Docker Engine running on
this virtual machine, run: docker-machine env do-worker-1

$ docker-machine create \
  --driver digitalocean \
  --digitalocean-access-token $DIGITAL_OCEAN_TOKEN \
  --digitalocean-region lon1 \
  do-worker-2
Running pre-create checks...
Creating machine...
«...»
Docker is up and running!
To see how to connect your Docker Client to the Docker Engine running on
this virtual machine, run: docker-machine env do-worker-2
```

We've now created two more machines in our DigitalOcean account, as we can verify by running: